💙

OO2 -FINAL

¿Qué es un patrón de diseño?

Según Alexander

> Un patrón describe un problema que ocurre una y otra vez, así como la solución a ese problema, de tal modo que se pueda aplicar esta solución muchas veces sin hacer lo mismo dos veces

- Un patrón de diseño es un par *problema-solución*
- Los problemas tratados por los patrones deben ser recurrentes, es decir, encontrarse con el mismo problema en distintas situaciones.
- La solución debe ser abstracta, que la persona interesada en leer esta solución comprenda

Según GOF (GangOfFour)

> Un patrón de diseño nomina, abstrae e identifica los aspectos clave de una estructura de diseño común, lo que nos hace útiles para crear un diseño orientado a objetos reutilizable. El patrón de diseño identifica las clases e instancias participantes, sus roles y colaboraciones, y la distribución de responsabilidades. Cada patrón de diseño se centra en un problema concreto, describiendo cuándo aplicarlo y si tiene sentido hacerlo teniendo en cuenta otras restricciones de diseño, así como las consecuencias y las ventajas e incovenientes de su uso.

Descripción de los patrones de diseño

- **Nombre:** Nombre del patrón, transmite su esencia.
- **Propósito/Intención:** Responde a las siguientes cuestiones: ¿Qué hace este patrón de diseño? ¿En qué se basa? ¿Cúal es el problema concentro de diseño que resuelve?
- **Motivación:** Un escenario que ilustra un problema de diseño y cómo las estructuras de clases y objetos del patrón resuelven el problema.
- **Aplicabilidad:** ¿En qué situaciones se puede aplicar el patrón de diseño? ¿Qué ejemplos hay de malos diseños que el patrón puede resolver?
- **Estructura:** Representación gráfica de las clases del patrón usando notación basada en Objetos.
- **Participantes:** Las clases y objetos participantes en el patrón de diseño, junto a sus responsabilidades
- **Colaboraciones:** Como colaboran los participantes en el patrón de diseño
- **Consecuencias:** ¿Como consigue el patrón sus objetivos? ¿Cuales son las ventajas y desventajas de su uso?
- **Implementación:** ¿Cuales son las dificultados, trucos o técnicas que deberíamos tener en cuenta a la hora de aplicar el patrón?
- **Código de ejemplo:** Fragmento de código que muestran cómo se puede implementar el patrón en C++ o Smalltalk
- **Usos conocidos:** Ejemplos del patrón en sistemas reales.
- **Patrones relacionados:** ¿Qué patrones estan muy relacionados con éste? ¿Cuales son las diferencias? ¿Con que patrones debería usarse?

Patrones Estructurales

Estos patrones tienen que ver con la composición de clases u objetos. Se ocupan de cómo se combinan las clases y los objetos para formar estructuras más grandes.

Adapter (Adaptador) - También conocido como Wrapper (Envoltorio)

※ **Propósito →**

Convierte la interfaz de una clase en otra interfaz distinta que es la que esperan los clientes. Permite que cooperen clases que de otra manera no podrían por tener interfaces incompatibles.

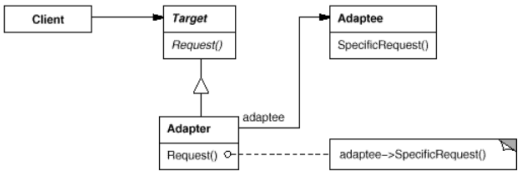
※ **Aplicabilidad →**

» Se quiere usar una clase existente y su interfaz no concuerda con la que necesita/no es compatible con lo que se precisa.

» Se requiere que clases con interfaces incompatibles puedan colaborar entre sí.

» Se quiere reutilizar una clase existente para que coopere con clases no relacionadas o que no han sido previstas (no fueron diseñadas para trabajar con dicha clase), es decir, que no tienen por qué tener interfaces compatibles.

※ **Estructura →**



※ **Participantes →**

- Target (Objetivo): Define la interfaz especifica del dominio que usa el Cliente. Es el objeto que recibe mensajes.

- **Client (Cliente):** Colabora con objetos que se ajustan a la interfaz Objetivo.

- **Adapter (Adaptador):** Adapta la interfaz de adaptable a la interfaz Objetivo. Funciona como pasa manos.

- **Adaptee (Adaptable)**: Define la interfaz existente que necesita ser adaptada.

Cada Adapter conoce a un Adaptee (relación 1 a 1) → Cuando se instancia un objeto concreto adaptado se crea una **instancia del Adapter**, y el Adapter queda asociado **uno a uno** con su Adaptee correspondiente. Por lo tanto, se crea un Adapter por cada objeto que se desea adaptar.

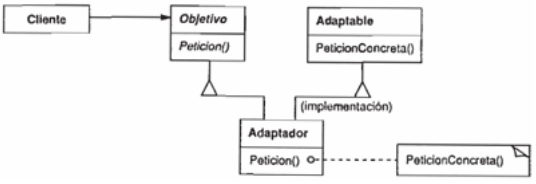
※ **Colaboraciones →**

- Los clientes llaman a operaciones de una instancia Adaptador. A su vez, el adaptador llama a operaciones de Adaptable, que son las que satisfacen la petición.
- El cliente se comunica con el Adapter a través de dicha interfaz y no conoce ni depende del Adaptee, puede enviar mensajes sin conocer la implementación concetra que los resuelve (esto es gracias al polimorfismo).

※ **Implementación →**

Se centra en convertir la interfaz de una clase en otra interfaz que esperan los clientes, permitiendo que clases incompatibles trabajen juntas. Existen dos formas de implementarlo:

1. **Implementación de adaptadores de clases:** Utiliza la **herencia múltiple** para adaptar una interfaz a otra.



Adaptador debería heredar públicamente de Objetivo (adopta su interfaz) y privadamente de la clase Adaptable (heredando su implementación).

Observación: No lo vemos en la materia pero es otra opción válida (no entre en detalle)

2. **Adaptadores conectables:** Se basa en la composición de objetos.

El adaptador mantiene un puntero o referencia a una instancia de la clase *Adaptable*. Lo que permite adaptar una clase y todas sus subclases, ya que adaptador puede apuntar a una instancia de la clase Adaptable o a cualquiera de sus derivadas.

Observación: Esta forma es la que vimos en la materia.

※ **Consecuencias →**

√ Flexibilidad con subclases: Permite que un mismo Adaptador funcione con muchos Adaptables; es decir; el Adaptable en sí y todas sus subclases (en caso de que las tenga).

√ El adaptador puede añadir funcionalidad a todos estos adaptables a la vez.

✕ Dificultad para redefinir comportamiento: Hace que sea más difícil redefinir el comportamiento de Adaptable. Si necesitas cambiar cómo funciona un método del objeto adaptado, no basta con heredar; necesitarás crear una subclase del `Adaptable` y hacer que el `Adaptador` se refiera a esa subclase en vez de a la original. (Esto se debe a que no se hereda de Adaptee y, por ende, no se puede sobreescribir métodos del Adaptee directamente).

Decorator (Decorador) - También conocido como Wrapper (Envoltorio)

※**Propósito →**

Asigna responsabilidades adicionales a un objeto dinámicamente, proporcionando una alternativa flexible a la herencia para extender funcionalidad.

※***Motivación - ¿Qué pasa con la herencia? →***

Se sabe que para agregar responsabilidades o comportamiento adicional a objetos se puede usar la herencia. Sin embargo, la herencia presenta limitaciones como:

↳ Las responsabilidades agregadas mediante la herencia se aplican a todas las instancias de una clase, no a objetos individuales.

↳ El uso de la herencia es estático: el cliente no puede decidir cuándo ni cómo se agregan o quitan responsabilidades.

↳ Si las responsabilidades deben cambiar dinámicamente, los objetos deberían cambiar de clase, lo cual no es viable.

↳ Con la herencia por cada responsabilidad adicional se crea una nueva clase. Esto da lugar a muchas clases diferentes y aumenta la complejidad del sistema.

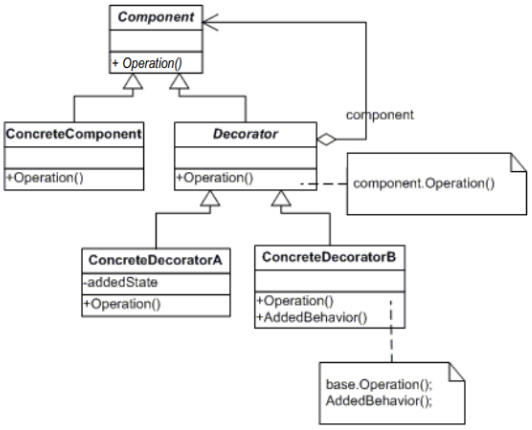
↳ Con la herencia, para añadir una propiedad dos veces, se debe heredar dos veces de una clase, lo cual es propenso a errores.

El patron Decorator surge para resolver este problema ya que *permite añadir o quitar responsabilidades dinámicamente a objetos individuales, sin modificar su clase*.

※**Aplicabilidad →**

- Para añadir comportamientos/responsabilidades (objetos individuales) a un objeto de forma dinámica y transparente, es decir, sin afectar a otros objetos.
- Para quitar responsabilidades dinámicamente.
- Cuando la extensión mediante herencia no es viable.
    - Si se quisiera un gran número de extensiones/variaciones independientes, se produciría una explosión de subclases para permitir todas las combinaciones de comportamiento. O puede ser que una clase esté oculta o no esté disponible para ser heredada.

※ **Estructura →**



※ **Participantes** →

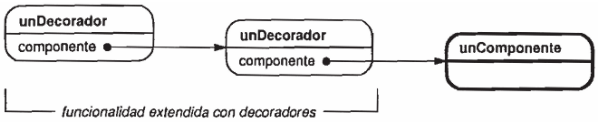
- Component (Componente): Define la interfaz para objetos a los que se puede añadir responsabilidades dinámicamente.
- ConcreteComponent (ComponenteConcreto): Define un objeto al que se le pueden añadir responsabilidades adicionales.
  - Un concreteComponent puede agregar nuevos datos además de cumplir con la interfaz Decorator.
- Decorator (Decorador): Tiene una referencia a un objeto Componente y define una interfaz que se ajusta a la interfaz del Componente.
- ConcreteDecorator (DecoradorConcreto): Añade responsabilidades al componente.

※ **Colaboraciones** →

- El Decorador mantiene una referencia al objeto Componente y redirige las peticiones a él.
- Antes y/o despúes de delegar las peticiones al objeto Componente, el Decorador puede realizar operaciones adicionales para agregar comportamiento adicional.
- Se encierra al Componente en otro objeto (Decorador) que añade la responsabilidad.
- El Decorador implementa la misma interfaz del Componente que decora de manera que su presencia es transparente a sus clientes. De esta manera, es posible anidar múltiples decoradores recursívamente, permitiendo así un número ilimitdo de responsabilidades añadidas.

※ **Implementación** →

- **Misma interface entre Component y Decorator:** Como la interfaz de un objeto Decorator debe ajustarse a la interfaz del componente que decora Component y, los ConcreteDecorator heredan de Decorator, estas clases también deben heredar de una clase común.
- **Omisión de la clase abstracta Decorator:** No siempre es necesario definir una clase abstracta
  - Si solo necesitas añadir una única responsabilidad, o si estás tratando con una jerarquía de clases existente (y no diseñando una nueva), se puede obtener la responsabilidad del Decorador directamente desde el ConcreteDecorator reenviando peticiones al Component en el ConcreteDecorator.
- **Mantener ligeras las clases Componente:** Esta clase debe centrarse en definir una interfaz y no en almacenar datos. Si la clase `Componente` es pesada (tiene muchos datos), las subclases decoradoras también lo serán, lo que añade un coste innecesario en rendimiento y complejidad
- **Cambiar la piel de un objeto en lugar de sus tripas:** Decorator sólo cambia la parte exterior de un objeto, el Componente no tiene que saber nada de sus Decoradores, es decir, los Decoradores son transparentes para el Componente.



※ **Consecuencias →**

√ **Más flexibilidad que la herencia estática:** Más flexibilidad para añadir responsabilidades a los objetos que la que puede obtenerse a través de la herencia estática.

√ Se pueden añadir y eliminar responsabilidades en tiempo de ejecución simplemente añadiendo o quitando decoradores.

√ Se pueden mezclar/combinar las responsabilidades, se define una clase simple y luego se añaden funcionalidades de manera incremental mediante objetos Decorator que envuelven al componente

√ Se puede añadir una misma propiedad dos veces con facilidad, añadiendo dos objetos decoradores de la misma clase.

√ **Evita clases cargadas de funciones en la parte alta de la jerarquía:** El Decorator permite definir clases sencillas y añadir funcionalidad de forma incremental con objetos decoradores. De esta manera, no se necesita pagar por funcionalidades que no se usan.

√ Esto vuelve facil definir nuevos Decoradores, independientemente de las clases de objetos de las que hereden, incluso para extensiones que no hubieran sido previstas. Es decir, se puede agregar nuevas funcionalidades SIN tocar la jerarquía de clases original aunque no se haya previsto cuando se diseño el sistema.

√ A diferencia de la herencia, donde extender una clase compleja tiende a exponer detalles no relacionados con las responsabilidades que estamos añadiendo, se agrega funcionalidad mediante composición sin exponer detalles internos de clases complejas.

× **Un decorador y su componente no son idénticos:** Un decorador actúa como una envoltura transparente, pero desde el punto de vista de la identidad de un objeto, un componente decorado no es idéntico al componente en sí. Por lo tanto, no deberías basar tu diseño en la identidad de los objetos cuando usas decoradores (por ejemplo, comparaciones por referencia)

```
if (obj == objetoOriginal) { ... }
//esto puede fallar si el cliente recibe el decorador,en lugar del objeto original
```

× Decorator suele dar como resultado sistemas formados por muchos objetos pequeños muy parecidos. Los objetos se diferencian en la forma que están interconectados. Los sistemas terminan siendo difíciles de aprender y depurar.

Proxy (Apoderado) - También conocido como Surrogate (Sustituto)

※ **Proposito →**

Proporcionar un representante o sustituto de otro objeto para controlar su acceso.

※ **Motivación →**

La razón de ser de este patrón surge cuando la creación e inicialización de un objeto es costosa y queremos retrasar ese costo hasta que el objeto sea realmente necesario (creación bajo demanda). El Proxy actúa como un intermediario que ocupa el lugar del objeto real. El cliente interactúa con el Proxy pensando que es el objeto real, y el Proxy se encarga de crear o cargar el objeto real solo cuando se solicita una operación que lo requiera estrictamente.

※ **Aplicabilidad →**

El patrón es aplicable siempre que se necesite una **referencia a un objeto** que sea más flexible o **sofisticada que un puntero simple**. Pueden distinguirse 3 casos en las que es aplicable el patrón:

» ***Virtual proxy:*** Se usa para demorar la construcción de un objeto hasta que esa realmente necesario. Cuando sea poco eficiente acceder al objeto real.

» ***Protection proxy:*** Se usa para restringir el acceso al objeto original, por seguridad.

» ***Remote proxy:*** Representa un objeto que se encuentra en un espacio de direcciones diferente (objeto remoto) en memoria local, es decir, es un representante “local” del objeto remoto que se encarga de comunicarse con él y de

serializar/deserializar los mensajes y resultados. Es la forma de implementar “objetos distribuidos”.

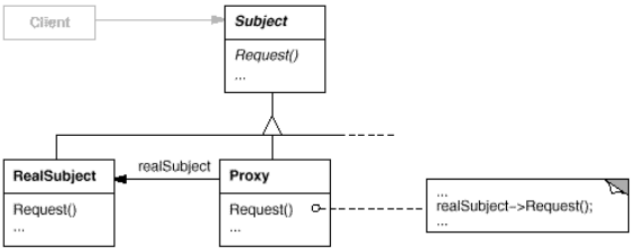
» El proxy empaqueta el request, lo envía a través de la red al objeto real, espera la respuesta, desempaqueta la respuesta y retorna el resultado.

» En este contexto el proxy suele utilizarse con otro objeto que se encarga de encontrar la ubicación del objeto real. Este objeto se denomina Broker.

esa estructura es igual para todos los tipos de Proxy (Virtual, Protection, Remote).

Lo que cambia NO es la estructura,
lo que cambia es la responsabilidad del Proxy.

※ **Estructura** →



※ **Participantes** →

- **Proxy (ProxyImagen):** Mantiene una referencia que le permite acceder al objeto real.
  - Proporciona una interfaz idéntica a la del `Sujeto`, de manera que un proxy pueda ser sustituido por el sujeto real.
  - Controla el acceso al `SujetoReal` y puede ser responsable de su creación y borrado
- **Sujeto (Grafico):** Define la interfaz común para el `SujetoReal` y el `Proxy`, permitiendo usar el Proxy en cualquier lugar donde se espere el `SujetoReal`
- **SujetoReal (Imagen):** Es el objeto verdadero que define la funcionalidad real

※ **Colaboraciones** →

El Proxy redirige las peticiones al `SujetoReal` cuando es necesario, dependiendo del tipo de proxy (por ejemplo, un proxy virtual esperará a redirigir hasta que el objeto real haya sido creado).

※ **Implemetación** →

- Proxy y RealSubject implementan la misma interfaz.
- Normalmente el cliente se relaciona con el proxy que luego será el encargado de crear/retornar el objeto real. Sin embargo, en un Virtual Proxy el cliente hace uso del proxy y luego pueden pasar dos cosas: el proxy sigue existiendo siempre o se elimina luego de crear el objeto real.

  En protection proxy el proxy no puede desaparecer ya que controla permisos siempre.
- El Proxy no siempre necesita conocer la clase concreta del `SujetoReal` si solo controla acceso, pero si debe instanciarlo (como en un Proxy Virtual), tendrá que conocer la clase concreta.

※ **Consecuencias** →

El patrón introduce un nivel de indirección al acceder a un objeto, lo cual genera las siguientes ventajas y desventajas

√ **Ocultamiento de ubicación:** Un proxy remoto puede ocultar el hecho de que un objeto reside en un espacio de direcciones o máquina diferente
√ **Optimizaciones:** Un proxy virtual permite realizar optimizaciones como crear objetos pesados solo cuando se necesitan, ahorrando recursos

√ Los proxies de protección permiten realizar tareas de mantenimiento adicionales cuando se accede a un objeto (por ejemplo, control de acceso, logging, carga diferida, etc. El cliente no tiene que preocuparse por nada de eso)
√ **Copy-on-write:** Permite implementar optimizaciones específicas como la "copia-de-escritura", donde la copia de un objeto complejo se posterga hasta que este es modificado, reduciendo el coste de operaciones de copiado.

Composite (Compuesto)

※ **Propósito →**

Compone objetos en estructuras de árbol para representar jerarquías de parte-todo.

> Estructuras donde objetos simples (partes) pueden agruparse para formar objetos compuestos (todo). Estos objetos compuestos pueden, a su vez, contener otros objetos simples o compuestos, formando estructuras recursivas.

Permite que los clientes traten de manera uniforme a los objetos individuales y a los compuestos.

> Los objetos simples (leaf) y los compuestos (composite) al compartir una misma interfaz permiten que, dentro del código, donde se pueda recibir un objeto primitivo también se pueda recibir uno compuesto.

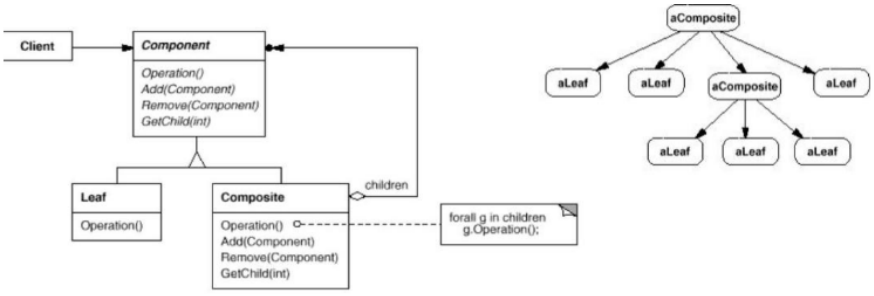
※ **Motivación →**

En aplicaciones gráficas (como editores de dibujo), los usuarios construyen diagramas complejos a partir de componentes simples (líneas, textos) y contenedores (grupos de elementos). El problema surge cuando el código debe tratar de forma diferente a los objetos primitivos y a los contenedores. Composite soluciona esto ya que define una clase abstracta que representa tanto a las primitivas como a sus contenedores, así, un objeto compuesto puede contener otros componentes (que a su vez pueden ser simples o compuestos), y el cliente puede ignorar estas diferencias.

※ **Aplicabilidad →**

- Se quiere representar jerarquías de objetos parte-todo
- Se quiere que los clientes ignoren las diferencias entre composiciones de objetos y objetos individuales. Los clientes deben tratarlos a todos de manera uniforme.

※ **Estructura →**



※ **Participantes →**

- **Component (Componente):**
  - Declara la interfaz para los objetos de la composicion
  - Implementa comportamientos default para la interfaz comun a todas las clases
  - Declara la interfaz para definir y acceder “hijos”.
  - (opcional) define una interfaz para para acceder el “padre” de un componente en la estructura recursiva y la implementa si es apropiado
- **Leaf (Hoja):**
  - Representa objetos “hojas” en la composición. Las hojas no tienen “hijos”.
  - Define el comportamiento de objetos primitivos en la composición.
- **Composite (Compuesto):**
  - Define el comportamiento para componentes con “hijos”.
  - Contiene las referencias a los “hijos”.
  - Implementa operaciones para manejar “hijos”.
- **Client (Cliente):** Manipula objetos en la composición a través de la interfaz Component.

※ **Colaboraciones →**

Los clientes usan la interfaz de la clase Componente para interactuar con los objetos de la estructura compuesta.

- Si el recipiente es una hoja, la petición se trata correctamente.
- Si es un compuesto, puede tener uno o muchos componentes hijos, por lo tanto, dirige el mismo mensaje que recibe a todos sus componentes hijos.

※ **Implementación** →

- **Referencias explícitas al padre:** Mantener referencias desde los hijos hacia el padre facilita el recorrido de la estructura y la gestión (borrado). El mejor lugar para definir esta referencia es en la clase `Componente` .
- **Transparencia vs seguridad:** *¿Donde conviene que esté definido el Add(Component), Remove(Component), GetChild(int)?*
  - La **transparencia** busca que el cliente pueda tratar de la misma manera a hojas y compuestos → Todas las operaciones (como Add, Remove, etc) se definen en la superclase común `Componente` y el cliente trata a todos los objetos igual, no necesita saber si está interactuando con una hoja o un compuesto, sin embargo, es inseguro ya que la hoja puede recibir mensajes sin sentido semántico (ej. añadir un hijo a una hoja).
  - La **seguridad** prioriza que los objetos solo expongan operaciones que tienen sentido → Todas las operaciones estructurales se definen solo en la clase `Compuesto`, las hojas no implementan métodos que conceptualemente no les corresponden, sin embargo, se pierde transparencia ya que el cliente debe distinguir entre hoja y compuesto.
- **Almacenamiento de hijos:** La clase `Compuesto` puede usar diversas estructuras de datos (listas enlazadas, vectores, tablas hash) para guardar a sus hijos.

❔ *¿Puede pasar un objeto de hoja a composite?*

No, un objeto no cambia su naturaleza (hoja o compuesto), lo que puede cambiar es cómo es utilizado dentro de la estructura.

Si se necesita tratar varias hojas como una unidad compuesta, se introduce un nuevo objeto Composite que las contiene. De esta manera, no se transforma la hoja en composite, sino que se agrega un nuevo nivel en la jerarquía que permite tratar a un conjunto de objetos como una sola entidad.

※ **Consecuencias** →

√ **Simplifica el cliente:** Los clientes pueden tratar a los objetos compuestos y a los individuales uniformemente, no deben preocupárse por ello. A su vez, el código del cliente se simplifica al evitar condicionales (tipo `if` o `switch`) para comprobar si un objeto es compuesto o no.

√ **Facilita añadir nuevos componentes:** Es tan simple como definir nuevas subclases de `Compuesto` o `Hoja` , funcionaran automaticamente con las estructuras existentes y el código cliente, sin necesidad de realizar cambios.

✕ La desventaja de facilitar la adición de componentes es que hace **difícil restringir qué componentes pueden formar parte** de una composición. El sistema de tipos no puede impedir que añadas un componente inadecuado; deben hacerse comprobaciones en tiempo de ejecución.

Patrones de Comportamiento

Los patrones de comportamiento tienen que ver con algoritmos (cómo se ejecutan las acciones) y con la asignación de responsabilidades a objetos (qué objeto se encarga de qué tarea). No solo describen patrones de clases, sino también patrones de comunicación entre ellos (cómo los objetos interactúan entre sí).

Template Method (Método plantilla)

※ **Propósito** →

Define en un metodo el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos.

Permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar la estructura.

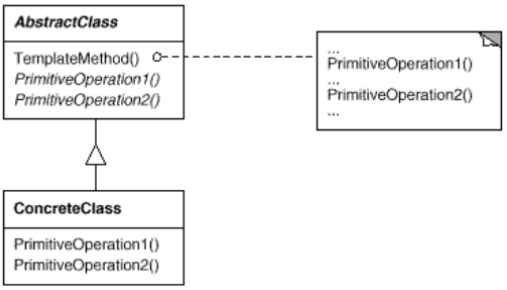
※ **Motivación** →

El patrón surge de la necesidad de definir el orden de los pasos de una operación en una clase base, pero permitir que las subclases implementen los detalles de esos pasos.

※ **Aplicabilidad** →

- Para implementar la partes invariantes de un algoritmo solo una vez y dejar que las sub-clases implementen los aspectos que pueden cambiar
- Cuando el comportamiento común de las subclases debería factorizarse y localizarse en una clase común para evitar la duplicación de código
- Se utilizan para implementar frameworks.
    - Puedes definir un método plantilla que llame a operaciones de "enganche" (*hooks*) en puntos específicos, permitiendo extensiones solo en esos puntos.

※ **Estructura** →



Siempre hay una jerarquia de clases. El template Method está en la superclas, que concentra la lógica en común y establece una secuencia fija de pasos, mientras que la variabilidad se expresa en los metodos primitivos que las subclases implementan según su comportamiento particular.

※ **Participantes** →

- **ClaseAbstracta:**
    - Implementa el método plantilla que define el esqueleto del algoritmo.
    - Define operaciones primitivas abstractas que las subclases concretas deben definir.
- **ClaseConcreta:**
    - Implementa las operaciones primitivas para realizar los pasos específicos del algoritmo, es decir, para expresar la variabilidad

※ **Colaboraciones** →

La `ClaseConcreta` se basa en la `ClaseAbstracta` para implementar los pasos invariantes del algoritmo.

※ **Implementación** →

- Control de acceso: Las operaciones primitivas que llama el método plantilla suelen declararse como miembros `protected`. Esto asegura que solo sean llamadas por el método plantilla

※ **Consecuencias** →
√ **Reutilización de código:** Es una técnica fundamental para reutilizar código. Permite "factorizar" el comportamiento común de varias clases y localizarlo en una clase padre común, evitando la duplicación de código en las subclases.

√ **Inversión de Control:** En lugar de que la subclase controle el flujo llamando a métodos de la clase padre, es la clase padre (la que contiene el método plantilla) la que llama a las operaciones de la subclase. Esto centraliza el control del algoritmo.

√ **Flexibilidad controlada mediante "Hooks":** Permite definir puntos específicos de extensión mediante *operaciones de enganche* (hooks). Estas operaciones proporcionan un comportamiento predeterminado en la clase padre y pueden ser redefinidas por las subclases si es necesario, pero no es obligatorio. Esto da flexibilidad sin obligar a implementar todo desde cero.

√ Garantiza que ciertos pasos críticos se ejecuten siempre en el orden correcto.

✕ Si el método plantilla llama a demasiadas operaciones primitivas que deben ser redefinidas, crear una subclase puede volverse una tarea "tediosa" para el cliente. Un objetivo clave de diseño debe ser **minimizar las operaciones primitivas** que una subclase está obligada a implementar

✕ Puede ser difícil para los usuarios de la clase distinguir qué métodos *deben* redefinir (operaciones primitivas abstractas) y cuáles *pueden* redefinir (hooks). (Se sugieren seguir convenciones)

✕ Hay restricción de acceso a las operaciones primitivas, deberían declararse como `protected` para asegurar que solo sean llamadas por la plantilla y no por externos.

🔎 **Funcionamiento (lookup)**

Es importante recordar como se invocan los mensajes (look-up) para entender el template method.

- Cuando un objeto recibe un mensaje, primero busca el método en su propia clase.
  - Si lo encuentra, lo ejecuta.
  - Si no lo encuentra, lo busca en la jerarquia. Se empieza a subir en la jerarquia hasta que se encuentra el método, se ejecuta como si fuera propio.

  Todas las referencias this dentro de ese método corresponden a la clase del objeto que recibió el mensaje originalmente, no a la clase donde se encontró el método → El lookup sube por la jerarquía pero el objeto no cambia.

❔ *¿Cual es la diferencia con el polimorfismo en sí?*

El template method es un patrón de diseño que utiliza el polimorfismo. El polimorfismo es una caracteristica de orientacion a objetos, donde objetos de clases distintas respondan al mismo mensaje de forma diferente.

Strategy (Estrategia) - También conocido como Policy (Política)

※ **Propósito** →

Define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables, es decir, se usan en forma intercambiada según se necesiten.

Permite que el algoritmo varíe independientemente de los clientes que lo usan.

※ **Motivación** →

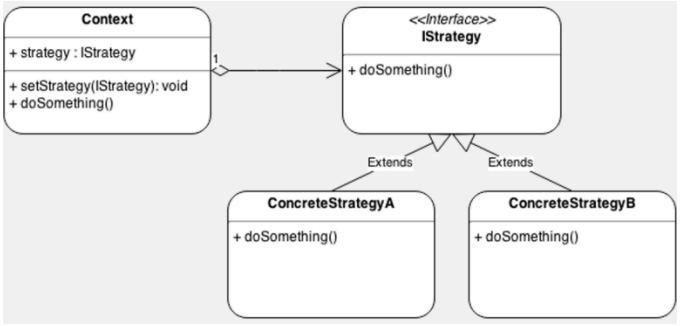
Existen muchos algoritmos para realizar una tarea específica. Codificar estos algoritmos directamente dentro de la clase que los usa trae problemas:

- El código del cliente se vuelve complejo/grande y difícil de mantener.
- Como los distintos algoritmos se usan en distintos momentos, es pesado cargar con algoritmos que quizás no se usen.
- El cliente se vuelve grande y pesado al cargar con algoritmos que quizás no use.

※ **Aplicabilidad** →

- Muchas clases relacionadas difieren solo en su comportamiento. Las estrategias permiten configurar una clase con un determinado comportamiento de entre muchos posibles.
  - Existen muchos algoritmos para llevar a cabo una misma tarea.
- Se necesitan variantes de un algoritmo.
- Una clase define muchos comportamientos y estos aparecen como múltiples sentendias condicionales (if/switch) en sus operaciones
  - No es deseable codificarlos todos en una clase y seleccionar cual utilizar por medio de sentencias condicionales.
- Si cada algoritmo utiliza información propia que los clientes no deberían conocer.

※ **Estructura** →



※ **Participantes →**

- **Strategy (Estrategia):** Declara una interfaz común a todos los algoritmos permitidos.

  El contexto usa esta interfaz para llamar al algoritmo definido por una EstrategiaConcreta.

- **ConcreteStrategy (EstrategiaConcreta)**: Implementa un algoritmo concreto, cumpliendo con la interfaz Strategy y define una forma especifica de realizar la operación.

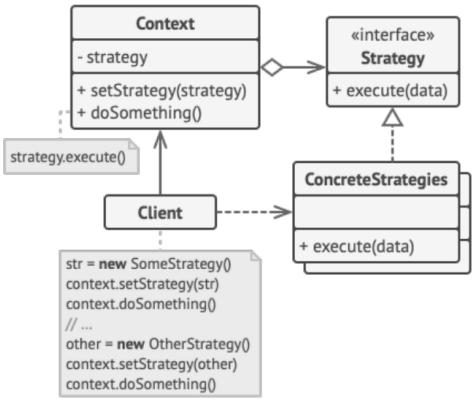
- **Context (Context):** Mantiene una referencia a un objeto Strategy, se configura con un objeto ConcreteStrategy.

※ **Colaboraciones →**

- Strategy y Context interactúan para implementar el algoritmo elegido. El Contexto puede pasar los datos requeridos como parámetros a la Strategy, o puede pasarse a sí mismo como argumento, permitiendo a la Strategy llamarle para obtener los datos que necesite.

- Los clientes configuran el Contexto con una `ConcreteStrategy` , es decir, crean un objeto ConcreteStrategy el cual luego pasan al contexto. A partir de ahí, el cliente interactúa con el Contexto, el contexto redirige las peticiones del cliente a su estrategia.

  De esta manera, los clientes interactuan exclusivamente con el contexto.



※ **Implementación →**

- Strategy y Context deben permitir a una Estrategia Concreta acceder de manera eficiente a cualquier dato que necesite del contexto y viceversa. Hay dos enfoques:

  1.1 ***Pasar datos como parámetros:*** Context pasa los datos como parámetros a las operaciones de Estrategia. Esto las mantiene desacoplados. Sin embargo, Context podría pasar datos a la Estrategia que ésta no necesita.

  1.2 ***Pasar el contexto:*** Context se pase a sí mismo como argumento y que la estrategia pida los datos explicitamente al contexto. Sin embargo, Context debe definir una interfaz más elaborada para sus datos, lo que acopla más estrechamente a Estrategia y Context.

- El Contexto puede tener un comportamiento predeterminado si no se le asigna ninguna estrategia (puntero nulo), simplificando su uso para clientes que no necesitan personalización

- Context debe tener en su protocolo métodos que permitan cambiar la estrategia.

※ **Consecuencias →**

√ Familias de algoritmos: Permite encapsular distintas variantes de algoritmos bajo una misma interfaz, son reutilizados por los contextos y facilita su intercambio dinamico. La herencia puede ayudar a sacar factor común de la funcionalidad de estos algoritmos.

> Permite definir distintos algoritmos que responden a un mismo método, en clases diferentes. Cada estrategia implementa ese método a su manera y, puede ser seleccionada o cambiada dinamicamente ya que el cliente crea la estrategia concreta y la pasa al Context, permitiendo cambiar el comportamiento en tiempo de ejecución (mediante el setStrategy que llama el cliente).

√ Alternativa a la herencia: Es una alternativa a subclasificar el contexto, para permitir que se pueda cambiar dinámicamente.

> En lugar de crear múltiples subclases del context para cada variante, se encapsula esos comportamientos en clases separadas, lo que permite cambiar el algoritmo dinámicamente, sin modificar ni heredar el contexto.

√ Desacopla al contexto de los detalles de implementación de las estrategias.

> El context trabaja con la interfaz Strategy, no con implementaciones concretas. El contexto no conoce como se implementa cada algoritmo.

√ Se eliminan los condicionales:Un código con muchas sentencias condicionales suele indicar la necesidad de aplicar este patrón.

> × Esto puede generar un pequeño costo adicional, se introducen más llamadas a métodos y hay más objetos intermediarios. Es decir, se puede generar overhead en la comunicación entre contexto y estrategias.

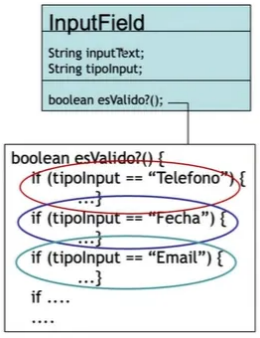
√ Permite elegir diferentes implementaciones del mismo algoritmo

× Los clientes deben conocer las diferentes estrategias para poder elegir la adecuada.

× Sobrecarga de comunicación: La interfaz de Estrategia es compartida por todas las clases de ConcreteStrategy, por lo tanto, puede pasar que algunos objetos ConcreteStrategy no usen toda la información que reciben a través de la interfaz (pueden incluso no utilizar nada). Algunas estrategias simples pueden recibir parámetros que no usan o tener que realizar llamadas costosas al contexto.

× Mayor número de objetos: Se incrementa la cantidad de objetos en la aplicación.

Deberia usarse solo cuando la variación de comportamiento sea relevante a los clientes.



State (Estado)- También conocido como Objects for States (Estados como Objetos)

※ **Propósito →**

Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno. Externamente parece que el objeto cambia de clase.

La idea es desacoplar el estado interno del objeto en una jerarquía de clases.

> En lugar de representar al estado como un valor interno dentro del objeto, se representan los estados como objetos separados, organizados en una jerarquia de clases.

※ **Motivación →**

El patrón surge cuando un objeto (como una `ConexionTCP` de red) puede estar en varios estados diferentes (Establecida, Escuchando, Cerrada) y debe responder de manera distinta a las mismas peticiones dependiendo de su estado actual. En lugar de llenar el código de grandes sentencias condicionales (`if` o `switch`) para comprobar el estado en cada operación, el patrón introduce una clase abstracta `EstadoTCP` y subclases para cada estado concreto. La clase `ConexionTCP` delega las operaciones dependientes del estado en el objeto de estado actual

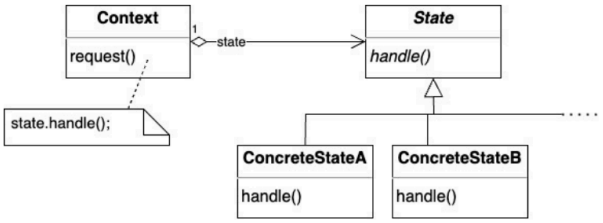
※ **Aplicabilidad** →

- El comportamiento de un objeto depende de su estado, y debe cambiar en tiempo de ejecución dependiendo de ese estado
- Los métodos tienen sentencias condicionales con múltiples ramas que dependen del estado, por ejemplo, si el estado es A hacer X, si es B hacer Y...

  El estado se representa usualmente por constantes enumerativas y el objeto consulta para decidir qué hacer.

El patrón State pone cada rama de la condición en una clase aparte, permitiendo tratar el estado como un objeto

※ **Estructura** →



※ **Participantes** →

- Context (Contexto):
  - Define la interfaz que conocen los clientes.
  - Mantiene una instancia de una subclase de ConcreteState que define el estado actual.
- State (Estado):
  - Define la interfaz para encapsular el comportamiento de los estados de Context
- ConcreteState (EstadoConcreto):
  - Cada subclase representa a un estado concreto en el que puede estar el objeto Context, implementa su comportamiento.

※ **Colaboraciones** →

- Contexto delega las peticiones que dependen del estado en el objeto EstadoConcreto actual.
  - Cuando Contexto recibe mensajes que dependen de su estado interno, los delega a la clase State concreta actual de él (polimorfismo)
- Un contexto puede pasarse a sí mismo como parámetro para que el objeto Estado acceda al contexto si fuera necesario para manejar la petición.
- Son los clientes quienes configuran el contexto con un objeto Estado inicial, pero después, tanto las subclases de Contexto como las de `EstadoConcreto` pueden decidir qué estado sigue a otro y bajo qué circunstancias

※ **Implementación** →

- **¿Quién define las transiciones?** El patrón no especifica si la lógica de transición debe estar en el Contexto o en las subclases de Estado. Si los criterios son fijos, pueden estar en el Contexto. Si son flexibles, es más conveniente que las propias subclases de Estado especifiquen su estado sucesor y cuándo realizar el cambio.
- **Creación y destrucción de objetos Estado:** Se puede optar por crear los objetos de estado solo cuando se necesitan y destruirlos después, o crearlos todos al principio y mantenerlos (útil si el cambio de estado es frecuente).
- **Uso de herencia dinámica:** En la mayor parte de los lenguajes de programación orientados a objetos, que no soportan cambiar la clase de un objeto en tiempo de ejecución, usan este patrón para simular dicha herencia

dinámica mediante la delegación y cambio de objeto delegado.

※ **Consecuencias →**

√ Localiza el comportamiento relacionado con cada estado.

El patrón situa todo el comportamiento asociado con un determinado estado en un solo objeto. Esto facilita añadir nuevos estados y transiciones, simplemente hay que definir nuevas subclases.

√ Las transiciones entre estados son explícitas.

El estado se representa con un objeto real, no es un valor simple (como un enum o un int).

La transición entre estados se convierte en una operación explícita, cambiar de estado implica cambiar un objeto completo (asignar un nuevo objeto a una variable) y no solo modificar variables internas.

```
context.setState(new EstadoA());
```

El cambio es atómico, el comportamiento del objeto cambia automáticamente cuando cambia su estado, evitando estados inconsistentes .

Además, la transición entre estados queda explícita en el diseño, se ven en la estructura del código.

√ Los objetos de estado pueden compartirse

Si los objetos Estado no tienen variables de instancia (solo comportamiento), pueden ser compartidos entre múltiples contextos. Esto se debe a que el estado no guarda información adicional, su comportamiento solo depende de qué clase es, no importa cúando ni para quién se use, siempre hace lo mismo. No hay diferencia entre crear uno nuevo o reutilizar uno existente.

Sin embargo, hay que tener en cuenta que, en el caso contrario, cada instancia puede ser distinta y si se comparten, se mezclarían datos de distintos objetos.

✕ Cuando las transiciones de estado se realizan entre las clases State concretas, hay bastante acoplamiento entre ellas ya que deben conocerse entre sí para gestionar las transiciones

Cada estado debe saber qué hacer y a qué estado pasar despúes, por lo tanto, cada estado debe conocer los posibles estados siguientes.

✕ Incrementa el número de clases: Al distribuir el comportamiento en varias subclases de Estado, aumenta el número de clases en el sistema.

Patrones Creacionales

Estos patrones tienen que ver con el proceso de creación de objetos. Buscan abstraer el proceso de creación de instancias, ayudando a que un sistema sea independiente de cómo se crean, componen y representan sus objetos.

Factory Method (Metodo de fabricación) - También conocido como Virtual Constructor (Constructor Virtual)

※ **Intención →**

Define una interfaz para crear un objeto, pero deja que las subclases sean quienes decidan qué clase instanciar. Permite que una clase delegue en sus subclases la creación de objetos.

¿Por qué se le dice Constructor Virtual?

Se lo denomina **constructor virtual** porque la creación del objeto no se hace mediante un constructor tradicional (new), si no que mediante un método polimórfico cuya implementación se decide en tiempo de ejecución.

※ **Motivación →**

Los frameworks suelen usar clases abstractas para definir y mantener relaciones entre objetos, pero también muchas veces son responsables de crear esos mismos objetos.

El problema surge cuando el framework (por ejemplo, un framework de aplicaciones que pueda presentar varios documentos al usuario) necesita crear objetos (Documento) pero no conoce ni puede predecir la clase concreta a

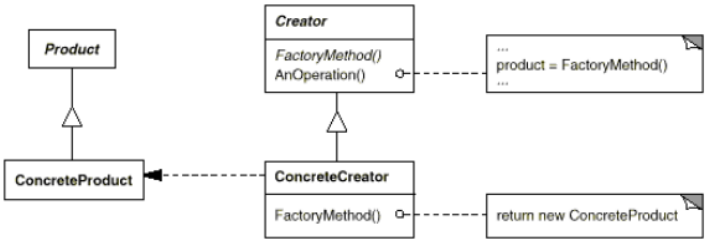
instanciar ya que depende de la implementación específica de la aplicación (ej. DocumentoDeDibujo, AplicaciónDeDibujo), solo sabe cúando debería crearse un nuevo objeto.

El patrón soluciona este problema encapsulando el conocimiento sobre qué subclase crear y sacando ese conocimiento fuera del framework, es decir, mueve la decisión de qué clase instanciar a las subclases.

※ **Aplicabilidad** →

- Cuando una clase no puede prever la clase de objetos que debe crear.
  - La clase sabe que necesita un objeto, pero no sabe cúal objeto concreto va a ser.
- Cuando una clase quiere que sean sus subclases quienes especifiquen los objetos que ésta crea.
- Las clases delegan la responsabilidad en una de entre varias clases auxiliares, y queremos localizar qué subclase de auxiliar concreta es en la que se delega.

※ **Estructura** →



La estructura implica una relación de herencia paralela donde los creadores se relacionan con los productos.

- Creator (Creador): Declara el método de fabricación, el cual devuelve un objeto de tipo Producto.
  - Puede definir una implementación predeterminada del método de fabricación que devuelva un objeto ProductoConcreto por defecto.
  - Puede llamar al método de fabricación para crear un objeto Product.
  - Necesariamente tiene subclases Creador Concreto.
- ConcreteCreator (Creador Concreto): Redefine el método de fabricación para devolver una instancia de un Producto Concreto.
- Product (Producto): Define la interfaz de los objetos que crea el método de fabricación
  - Puede tener subclases Producto Concreto o no
- ConcreteProduct (Producto Concreto): Implementa la interfaz Producto

※ **Colaboraciones** →

El Creador se apoya en sus subclases (ConcreteCreator) para definir el método de fabricación de manera que éste devuelva una instancia de un Producto Concreto.

El patrón requiere un protocolo polimórfico: el Creador define el metodo de fabricación pero no lo implementa, todas las subclases ConcreteCreator responden/implementan el mismo metodo, aunque cada una instancia un producto diferente.

Los productos comparten una interfaz común (Product), y el Creator trabaja contra esta abstracción en lugar de depender de clases concretas, lo que permite desacoplar la creación del uso.

※ **Implementación** →

- Hay dos variantes principales
  - El caso en que la clase `Creador` es abstracta y no proporciona implementación para el método de fabricación por defecto. Esto requiere que las subclases definan una implementación porque no hay ningún comportamiento predeterminado.
  - El caso en que el `Creador` es una clase concreta y proporciona una implementación predeterminada del método de fabricación. Las clases `Creador Concreto` usa el metodo por flexibilidas, pueden cambiar la clase de objetos que

instancia su clase padre si es necesario.

- Métodos parametrizados: El método de fabricación puede aceptar un parámetro (como un ID o un tipo) para crear diferentes tipos de productos. Esto permite que un solo `Creator` produzca diferentes `Products`
- Uso de plantillas (Templates): Se puede usar para evitar la herencia obligatoria, pasando la clase del producto como parámetro de plantilla a la clase `Creator`

※ **Consecuencias** →

√ **Elimina la necesidad de ligar clases específicas:** El código solo trata con la interfaz `Product`, por lo que puede funcionar con cualquier clase `ConcreteProduct` definida por el usuario.

√ **Proporciona puntos de extensión (hooks) para las subclases:** Crear objetos mediante un método de fabricación es más flexible que hacerlo directamente con un constructor. El patrón proporciona a las subclases un punto de extensión (hook) para que redefinan la creación del objeto y devuelvan una versión especializada, sin modificar el código del creador base.

√ **Conecta jerarquías de clases paralelas:** Una jerarquia de clase paralela se produce cuando una clase delega alguna de sus responsabilidades a una clase separada. Cuando hay dos (o má) árboles de herencia distintos, pero las clases de uno de los árboles tienen una correspondencia uno-a-uno con las clases del otro árbol.

El metodo de fabricación define la conexión entre las dos jerarquías de clases, localizando qué clases van juntas.

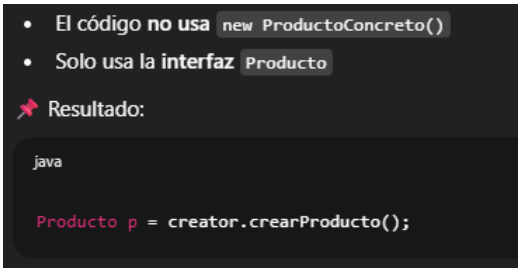
✕ El cliente puede verse obligado a heredar de la clase Creador: Los clientes pueden tener que heredar de la clase Creador simplemente para crear un determinado objeto ProductoConcreto, lo que puede complicar la jearaquía si no se necesitaba heredar por otra razón.

Si el cliente tenia que heredar de todos modos de la clase Creador, esta desventaja es menos relevante.

✕ El patrón introduce jerarquías y puede aumentar el número de clases significativamente: Por cada nuevo producto se requiere una nueva clase ConcreteProduct y un nuevo ConcreteCreator

¿Cuando vale la pena Factory Method?

- Cuando se necesite abtraer la construcción de un objeto y delegar la decisión de qué objeto concreto crear a las subclases
  - No hay acoplamiento entre el objeto que necesita un objeto y el objeto creado (la clase que usa el producto trabaja con una interfaz, no depende de la clase concreta del producto)



- Cuando una clase no puede anticipar qué tipo concreto de objeto deberá crear.
  - La creación se resuelve mediante polimorfismo en tiempo de ejecución.
- Cuando se quiere localizar la lógica de creación en un solo lugar y mantener el código más limpio.
- Cuando hay que facilitar agregar al sistema nuevos tipos de productos.
  - Cada producto solo requiere un nuevo ConcreteCreator, no se requiere modificar código existente.
- Cuando aparecen múltiples condicionales que determinan qué objeto instanciar.

¿Cuando no vale la pena Factory Method?

- Cuando la creación es simple y directa
  - El patrón introduce complejidad en el codigo (más clases y niveles de abstracción)

- 1 llamada a un constructor vs diseñar e implementar varios roles.

Observación: El patrón Factory Method suele aparecer dentro de un **Template Method** (aunque no es obligatorio).

Existe una clase base con un método principal que define un flujo (template method). Dentro de ese flujo, se llama a un método fábrica que decide qué objeto crear. Las subclases redefinen ese método fábrica.

```
abstract class Dialog {

    // Template Method
    public void render() {
        Button okButton = createButton(); // factory method
        okButton.draw();
    }

    // Factory Method
    abstract Button createButton();
}
```

//Acordate el ejemplo de polizas.

Builder (Constructor)

※ **Intención →**

Separa la construcción de un objeto complejo de su representación, de forma que el mismo proceso de construcción pueda crear diferentes representaciones.

※ **Motivación →**

El libro utiliza el ejemplo de un lector de formato de intercambio de documentos RTF (*Rich Text Format*). Una aplicación necesita leer un documento RTF y convertirlo a diferentes formatos: texto ASCII puro, TeX, o un objeto de interfaz de usuario editable. El problema es que el número de formatos de conversión es indefinido y puede crecer. Si codificamos la lógica de conversión directamente en el lector, el código se vuelve rígido y difícil de extender. La solución del patrón Builder es separar el algoritmo que interpreta el formato (el **Director**) del código que crea la representación específica (el **Constructor**). El lector simplemente llama a métodos como `ConvertirCaracter` o `ConvertirParrafo` en una interfaz abstracta `Constructor`, y las subclases de este (`ConstructorASCII`, `ConstructorTeX`) se encargan de los detalles específicos

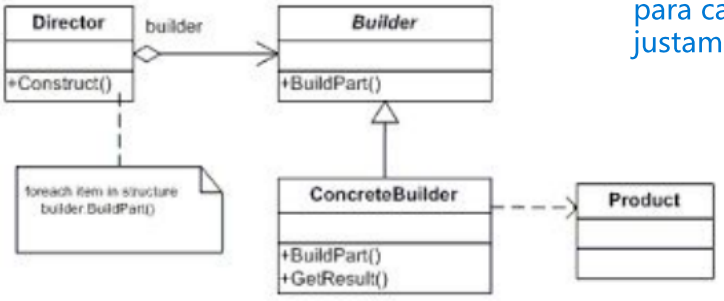
※ **Aplicabilidad →**

- Cuando el algoritmo, para crear un objeto complejo, deberia ser independiente de las partes de que se compone dicho objeto y de cómo se ensamblan.
- Cuando el proceso de construcción debe permitir diferentes representaciones del objeto que está siendo construido.

※ **Estructura →**

ojo que vos querias hacer subclases para cada sandwich y eso es justamente lo que evita este patron

el builder puede tener un metodo reset para reutilizar el mismo builder



en realidad el builder tiene el product, todos heredan esa relación

Un cliente configura un objeto Director con una instancia de ConstructorConcreto. El Director guia el orden del proceso de construcción invocando a métodos del Constructor (Builder). El Constructor (Builder) es él que sabe construir cada parte y termina armando el Producto.

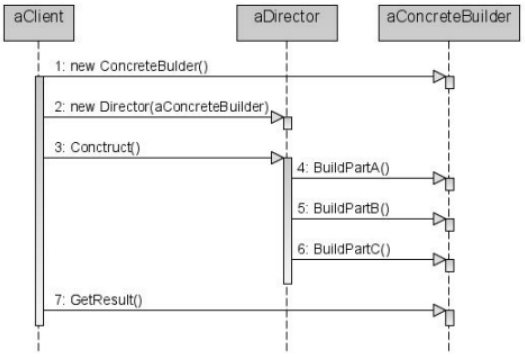
La jerarquia de builders debe ser polimorfica, todos los builders comparten una interfaz común. El director nunca consulta el tipo del builder.

※ **Participantes** →

- Builder (Constructor): Especifica una interfaz abstracta para crear las partes de un objeto Producto
- Concrete Builder (Constructor Concreto): Implementa la interfaz Constructor (Builder) para construir y ensamblar las partes del producto.
  - Define la representación a crear, es decir, define qué producto final va a resultar del proceso de construcción.
  - Construye la representanción interna del producto y define el proceso de ensamblaje.
  - Proporciona una interfaz para devolver el producto (tiene un método que devuelve el producto)
  - Puede implementar más metodos que los utilizados por el Director. El Director usa solo una parte de la interfaz (esto es porque el Director representa UN proceso específico, una secuencia particular de pasos, y un Builder puede usado por distintos Directores o por clientes que requieran procesos diferentes, etc.)
- Director: Construye un objeto usando la interfaz Constructor (Builder). Solo sabe hacer una cosa, qué construir y en que secuencia hacerlo, no necesita conocer el Producto.
- Product (Producto): Representa el objeto complejo en construcción.
  - Incluye las clases que definen sus partes constituyentes, incluyendo interfaces para ensamblar las partes en el resultado final. Es decir, el producto esta formado por piezas y tiene formas de ir agregando esas partes.

※ **Colaboraciones** →

- El cliente crea el objeto Director y lo configura con el objeto Constructor deseado.
- El Director notifica al constructor cada vez que hay que construir una parte de un producto.
- El constructor maneja las peticiones del director y las añade al producto.
- El cliente obtiene el producto del constructor.



※ **Implementación** →

- Hay un clase abstracta `Builder` que define una operación para cada componente que puede ser creado. Estas operaciones, por defecto, no hacen nada. Las subclases `ConstructorConcreto` redefinen las operaciones para los componentes que está interesado en crear.
- **Interfaz de ensamblaje y construcción:** Los constructores construyen sus productos paso a paso, es decir, añaden partes al producto paso a paso a través de una interfaz común. Esta interfaz debe ser lo suficientemente general como para permitir la construcción de productos por parte de todos los tipos de constructores concretos.
- ¿Por que no usar clases abstractas para los productos? Los productos creados por los distintos constructores tienen representaciones tan variadas que no tendría sentido definir una clase padre común para ellos.
- Métodos vacíos de manera predeterminada en el constructor: En lenguajes como C++, es común que los métodos de la clase `Constructor` se declaren con cuerpos vacíos, para que el cliente solo tenga que implementar las operaciones específicas que necesita.

※ **Consecuencias →**

√ Permite variar la representación interna de un producto: El objeto `Constructor` proporciona al `Director` una interfaz abstracta para construir el producto. Esta interfaz permite que el `Constructor` oculte la representación y la estructura interna del producto. Dado que el producto se construye a través de una interfaz abstracta, para cambiar su representación interna solo hay que definir un nuevo tipo de constructor.

√ Aísla/separa el código de construcción y representación: Se mejora la modularidad al encapsular la forma en que se construye y se representa un objeto complejo. Los clientes no necesitan saber nada de las clases que definen la estructura interna del producto; dichas clases no aparecen en la interfaz del Constructor.

√ Cada ConstructorConcreto tiene el código para crear y ensamblar un determinado tipo de producto. Este código sólo se escribe una vez y, despúes, los diferentes directores pueden reutilizarlo para construir variantes de Producto a partir del mismo conjunto de partes

√ Proporciona un control más fino sobre el proceso de construcción: Builder construye el producto paso a paso, bajo el control del director. La interfaz Director solo obtiene el producto del constructor una vez que éste está terminado.

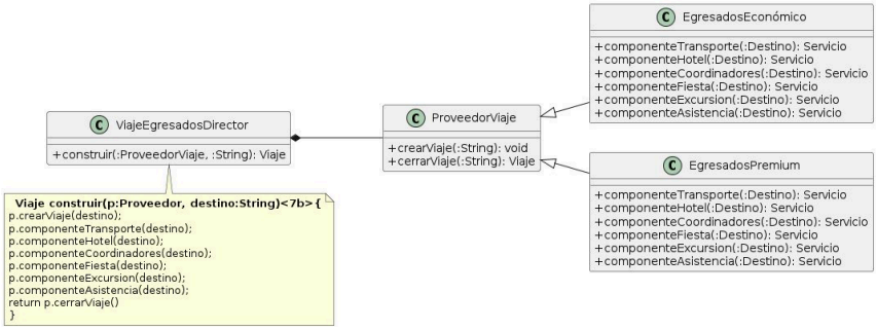
¿Cuando vale la pena usarlo?

- Cuando se necesita abstraer la construcción de un objeto complejo.
- Cuando se quiere variar lo que se construye usando el mismo proceso. Un mismo Director puede generar resultados distintos simplemente cambiando el Builder y también distintos Directores pueden reutilizar el mismo Builder con procesos distintos.
- Cuando se necesita controlar y ordenar los pasos de construcción. El director define la secuencia del armado y garantiza que el proceso siga un flujo específico.

  👉 A diferencia de Factory Method, donde solo se delega la creación del objeto, aquí se controla todo el proceso paso a paso

¿Cuando no vale la pena usarlo?

- Cuando la construcción es simple. Este patrón requiere diseñar e implementar varios roles. Si tenemos un problema donde, por ejemplo, solo existe una forma de construir el producto no valdra la pena ya que seria mucho trabajo para resolver algo más simple, deberia haber al menos más de un builder.
- Aumenta la cantidad de clases. Cada variación del producto suele requerir un ConcreteBuilder.
- Agrega complejidad conceptual en sistemas donde los builders actúan como parsers o ensambladores de especificaciones.



Preguntas de Final...

❔ ***¿Como se implementa un viaje a Punta del Este?***

No habria que cambiar nada. Hay que asegurar que en la base de datos se encuentre toda la información necesaria para poder elegir la opción de un destino a Punta del Este, esto es independiente y está por fuera del patrón.
El patrón Builder se ocupa del *cómo se construye*, no del contenido de la base de datos.

❔ ***¿Cómo se implementa un viaje a Rio de Janeiro con aéreo y traductor?***

La incorporación del servicio de traductor implica agregar un nuevo paso dentro del proceso de construcción del viaje "construir" en el Director. Dado que el Director representa un algoritmo específico de construcción, lo más adecuado sería definir un nuevo Director que incluya la invocación del método "componenteTraductor(destino)".

Recordar: Un director debe saber hacer una sola cosa.

Aunque un Builder puede implementar métodos adicionales no utilizados por el Director, en este caso el nuevo servicio requiere ser invocado explícitamente durante la construcción, por lo que modificar o crear un Director resulta necesario.

Se debe crear un nuevo builder concreto para pasaje aereo en lugar de colectivo y que responda al mensaje correspondiente al traductor.

❔ ***¿Cómo se implementa un viaje con múltiples destinos?***

Esto afecta la estructura que teniamos, el método actual del Director está diseñado para construir un viaje con un único destino. Si el soporte de múltiples destinos implica modificar el flujo de construcción, por ejemplo:

-Iterar destinos como "for destino in destinos: transporte (destino) ...asistencia(destino)").

-podria modelarse dentro del director los pasos de componenteTransporte, componenteHotel, ..., componentreAsistencia para el primer destino, despúes lo mismo para el segundo destino, y asi...

quería conveniente introducir un nuevo Director que represente este nuevo algoritmo de construcción.

¿Es necesario nuevos builders? no implica necesariamente la creación de nuevos Builders, ya que los Builders existentes pueden reutilizarse siempre que sepan construir los componentes individuales para cada destino. Solo sería necesario un nuevo Builder si cambia la forma de construir las partes del producto.

The Null Object Pattern - También conocido como Stub, Active Nothing

※ **Intención →**

Proporcionar un **sustituto para otro objeto** que comparte la misma interfaz pero que **no haga nada.**

Este objeto nulo encapsula las decisiones de implementación sobre cómo "no hacer nada" y oculta esos detalles de sus colaboradores.

※ **Motivación →**

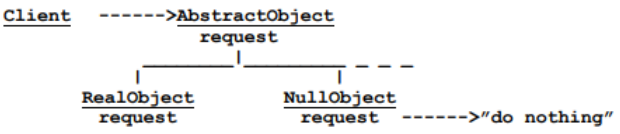
A veces una clase que, requiere un colaborador, necesita tratar a uno que no hace nada de la misma manera que a uno que provee un comportamiento.

Tradicionalmente, esto se resolvía usando `nil` (o nulo), pero esto obligaba al cliente a realizar comprobaciones constantes mediante condicionales que verifican la existencia de un objeto (`if objeto != nil`) antes de enviar mensajes, lo que ensucia el código, dificulta su reutilización y aumenta la complejidad ya que hay comportamiento que depende de la existencia o no existencia de un objeto. El patrón permite que el cliente envíe mensajes de forma uniforme sin preocuparse por la existencia o no del colaborador.

※ **Aplicabilidad →**

- Un objeto **requiere un colaborador** (el patrón no introduce la colaboración, utiliza una colaboración ya existente).
- Algunas instancias de los colaboradores **no deben hacer nada**.
- Se desea que los clientes **ignoren la diferencia** entre un colaborador con comportamiento real y uno que no hace nada, evitando comprobaciones explícitas de `nil.`
- Se busca **reutilizar el comportamiento de "no hacer nada"** para que diversos clientes que lo necesiten trabajen de forma consistente (de la misma manera).

※ **Estructura →**



El NullObject implementa la misma interfaz que el objeto real pero con métodos que no ejecutan acciones.

※ **Participantes** →

- **Client (Cliente):** La clase que requiere y utiliza al colaborador.
- **AbstractObject (Objeto Abstracto):** Define la interfaz para el colaborador del cliente y puede implementar comportamiento por defecto.
- **RealObject (Objeto Real):** Define una subclase concreta que proporciona el comportamiento útil esperado por el cliente.
- **NullObject (Objeto Nulo):** Provee una interfaz idéntica a la de AbstractObject para que un objeto nulo pueda ser sustituido por un RealObject.
    - Implementa su interfaz para no hacer nada. Lo que significa “no hacer nada” es subjectivo y depende del tipo de comportamiento que espera el cliente. Puede no hacer literalmente nada o devolver un resultado nulo.
    - Cuando hay más de una form de no hacer nda, se puede requerir más de una clase NullObject.

※ **Colaboraciones** →

Los clientes interactúan con sus colaboradores exclusivamente a través de la interfaz de la clase abstracta.

- Si el receptor es un **RealObject**, la solicitud se maneja con un comportamiento real.
- Si el receptor es un **NullObject**, la solicitud se maneja sin hacer nada o proporcionando un resultado nulo, de modo que el cliente no percibe la diferencia en la forma de invocar la operación.

※ **Implementación** →

- **Como Singleton:** Dado que los objetos nulos normalmente no tienen estado y este no cambia, se suele usar una única instancia compartida para todo el sistema.
- **Instancia especial de objeto real:** Para evitar crear muchas subclases, el objeto nulo puede ser una instancia especial del objeto real con sus variables configuradas en valores nulos.
- **No es un Proxy:** A diferencia de un Proxy, que controla el acceso a un objeto real, el Null Object **reemplaza** al objeto real y nunca muta para proporcionar comportamiento real.
- Uso como Strategy o State: Puede verse como un caso especial de los patrones Strategy o State, donde la estrategia o el estado actual es, precisamente, "no hacer nada"
- No es un Mixin: Un "mixin" es un mecanismo para agregar comportamiento a una clase mediante la mezcla de fragmentos de código. Null Object no funciona agregando código de "no hacer nada" directamente dentro de la clase del cliente, sino que define una clase colaboradora concreta e independiente. El Null Object es una clase real que implementa una interfaz específica (`AbstractObject`) para que pueda ser utilizada en cualquier lugar donde el cliente espere un objeto de ese tipo. El cliente no "se vuelve" nulo; simplemente **interactúa con un objeto externo** que ha sido diseñado para no realizar ninguna acción

※ **Consecuencias** →

√ **Simplifica el código del cliente:** Los clientes pueden tratar a los objetos reales y nulos de manera uniforme. Los objetos nulos pueden usarse en lugar de objetos reales cuando se espera que el objeto no haga nada. Siempre que el código del cliente espere un objeto real, también puede aceptar un objeto nulo.

Los clientes normalmente no saben (y no deberian preocuparse) si están tratando con un colaborador real o nulo. Esto elimina la necesidad de tener en el código condicionales que verifican si la referencia a un objeto es NULL.

√ **Encapsulamiento:** Centraliza el código de "no hacer nada" en el objeto nulo, es fácil de encontrar, de mantener y asegura que todos los clientes lo usen de la misma forma.

√ **Reutilización:** El comportamiento nulo puede ser compartido por múltiples clientes. Si es necesario modificar el comportamiento de inactividad, el código se puede cambiar en un solo lugar.

√ Hace explícitos los elementos del dominio cuyo comportamiento es 'no hacer nada', representándolos mediante un objeto específico.

× **Explosión de clases:** Puede requerir la creación de una nueva clase `NullObject` por cada nueva clase abstracta `AbstractObject.`

× **Dificultad de distribución:** No es fácil mezclar comportamientos nulos en objetos que ya tienen lógica propia. No se puede simplemente inyectar un pedacito de “no hacer nada” dentro de cada clase sin modificar su comportamiento.

× **Dificil de implementar:** Si varios clientes no están de acuerdo sobre cómo el objeto nulo debe no hacer nada,

× **Inmutabilidad funcional:** Un objeto nulo **siempre** actúa como tal; si el objeto necesita empezar a realizar acciones reales en algún momento, entonces no es un objeto nulo (podría ser un Proxy o un objeto con un modo de "solo lectura")

Type-Object

※ **Intención →**

Desacoplar las instancias de sus clases para que estas clases se puedan implementar como instancias de una clase.

Traducción: Identificar elementos del dominio que actúan como clases/metadata de otros elementos del dominio, es decir, son conceptos que le dan a otros objetos del dominio una caracteristica de inclusión.

Type-Object permite la creación dinámica de nuevas “clases” dinamicamente en tiempo de ejecución, permite que el sistema proporcione sus propias reglas de comprobación de tipos y puede conducir a sistemas más simples y pequeños.cxd3

※ **Motivación →**

A veces una clase no solo requiere un número desconocido de instancias, si no también un número desconocido de subclases. En los sistemas orientados a objetos tradicionales, aunque crear instancias es dinámico, crear nuevas clases usualmente requiere recompilación.

En lugar de tener una clase y crear un número desconocido de subclases, se transforma el diseño para que las variaciones se representen como instancias (objetos tipo), y así pasar a tener una clase con un número desconocido de instancias.

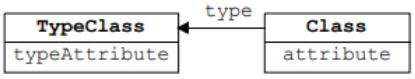
```
//Normalmente, cuando tenemos tipos distintos hacemos esto: cada tipo requiere una sub
clase
class Enemy
class Zombie extends Enemy
class Vampire extends Enemy
class Alien extends Enemy
-> //Despúes (con type object), en lugar de usar subclases, convierte los tipos en obj
etos reales
class Enemy //tendra referencia al tipo EnemyType, ej. "type"
class EnemyType
//se crean OBJETOS que representan tipos
EnemyType zombie = new EnemyType("Zombie");
EnemyType vampire = new EnemyType("Vampire");
EnemyType ghost = new EnemyType("Ghost");
//estos son los
TypeObject("Zombie")
TypeObject("Vampire")
TypeObject("Ghost")
//Ahora las instancias apuntan al tipo:
Enemy e1 = new Enemy();
e1.type = zombie;
Enemy e2 = new Enemy();
e2.type = zombie;
Enemy e3 = new Enemy();
e3.type = vampire;
//ahora el tipo no esta en una clase, está en un objeto
```

※ **Aplicabilidad →**

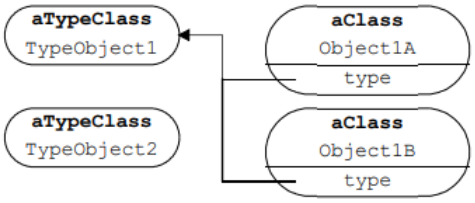
- Las instancias de una clase necesitan agruparse según atributos o comportamientos comunes. Existen conceptos del dominio que definen características comunes a muchas instancias.

- Se necesita una subclase para cada grupo para implementar los atributos/comportamientos comunes de dichos grupo.
- La clase requiere un gran número de subclases y/o la variedad total de subclases requerida es desconocida.
- Se necesita crear nuevos grupos en tiempo de ejecución que no fueron previstos durante el diseño.
- Se necesita cambiar la subclase de un objeto despúes de instanciarlo sin tener que mutarlo a una nueva clase.
- Se necesita anidar grupos recursivamente para que un grupo sea en sí mismo un elemento de otro grupo

※ **Estructura** →



El patrón Type-Object tiene dos clases concretas: una que representa objetos Class (ej. Enemy) y otra que representa sus tipos TypeClass (ej.EnemyType). Cada objeto Class tiene un puntero (referencia) a su tipo correspondiente.



El sistema usa un TypeObject para representar cada tipo en el sistema y un Object para representar cada instancia de esos TypeObject. Cada Object tiene un puntero (referencia) a su TypeObject.

※ **Participantes** →

- TypeClass: Es la clase del `TypeObject`. Tiene una instancia separada para cada tipo de objeto `Object`.
- TypeObject: Es una instancia de `TypeClass`.
  - Representa un tipo de `Object`. Establece las propiedades de un Object que son iguales para todos los objetos Object de ese mismo tipo.
- Class: Es la clase del `Object`. Representa las instancias de TypeClass
- Object Es una instancia de `Class`.
  - Representa un ítem único con un contexto único.
  - Mantiene un puntero a un `TypeObject` que describe su tipo. Delega en él las propiedades definidas por su tipo.

※ **Colaboraciones** →

- Un `Object` recibe dos categorías de peticiones: las definidas por su instancia y las definidas por su tipo. Maneja las de instancia por sí mismo y **delega** las peticiones de tipo a su `TypeObject`.
- Los clientes pueden interactuar directamente con los `TypeObjects`
- Si es necesario, el `TypeObject` puede tener un conjunto de punteros a sus `Objects`. De esta manera, el sistema puede recuperar fácilmente un Object que se ajuste a su descripción de TypeObject.

※ **Implementación** →

- Referencias: Cada `Object` debe tener una referencia a su `TypeObject`, la cual debe especificarse al crear el objeto.
- Comportamiento: El `Object` decide qué delegar. A menudo, al delegar, el `Object` pasa una referencia de sí mismo al `TypeObject` para que este pueda acceder a sus datos.
- No es herencia múltiple: La `Class` (no el `TypeObject`) es la plantilla para el nuevo `Object`. El `Object` no hereda los mensajes del `TypeObject`; debe delegarlos explícitamente.

- Creación de objetos: A menudo, un nuevo `Object` se crea enviando una petición al `TypeObject` apropiado (el `TypeObject` actúa como una fábrica).
- Cambio dinámico: Permite cambiar la "clase" de un objeto dinámicamente simplemente cambiando su puntero a un `TypeObject` diferente. Esto es mucho más simple que mutar un objeto a una nueva clase

※ **Consecuencias** →

√ Creación de clases en tiempo de ejecución: Permite crear nuevas "clases" (TypeObjects) dinámicamente sin recompilar.

√ Evita la explosión de subclases: Reduce drásticamente la cantidad de clases necesarias en el sistema.

√ Cambio de tipo dinámico: Facilita cambiar el comportamiento de un objeto en tiempo de ejecución asignándole otro TypeObject.

√ Subclasificación independiente: Permite que `TypeClass` y `Class` tengan sus propias jerarquías de herencia de forma independiente.

× Complejidad de diseño: Divide un objeto lógico en dos clases. La relación "cosa y su tipo" puede ser confusa para modeladores y programadores, dificultando la comprensión del sistema.

× Complejidad de implementación: Las diferencias de implementación se mueven de las subclases al estado de las instancias de `TypeObject`. Ya no se pueden implementar métodos diferentes para cada tipo mediante polimorfismo de clases tradicional; el `TypeObject` debe manejar las variaciones mediante su estado.

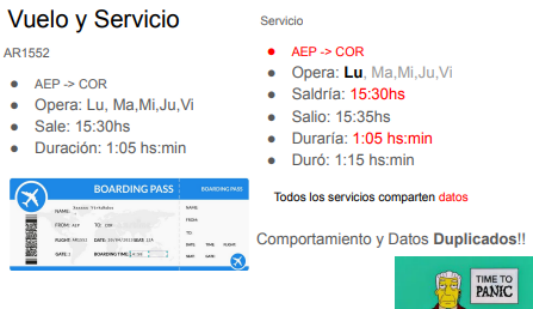
× Gestión de referencias: El sistema no mantiene automáticamente la relación clase-instancia entre el `TypeObject` y el `Object`; la aplicación debe gestionar manualmente esta referencia

//Ejemplo video

En el siguiente dominio hay cosas distintas que al principio parecen lo mismo, pero no lo son.

Vuelo: No es un evento concreto, es una definición. Tiene cosas que nunca cambian entre instancias, información compartida, como que opera de lunes a viernes, horario planificado de salida, etc. Describe el dominio.

Servicio: Es una ocurrencia concreta, es un vuelo en un día particular con un horario de salida que dependerá del retraso, etc.



El servicio es un objeto que depende del vuelo, hereda un montón de datos del vuelo, los “conceptos del dominio”. Si cada servicio guardara todo, se estaría duplicando datos constantemente.

Solución: Separar lo que define el tipo de lo que es una instancia concreta.

Servicio será una clase y se crearan instancias de servicio. Estará relacionada (tendra una referencia) con un vuelo.

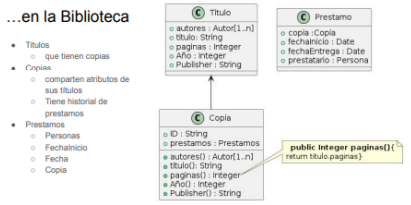
El servicio definirá todo lo que el vuelo necesita como dominio. Las cosas que quedan propias del vuelo se las pedira a la clase vuelo, no las volvera a guardar, evitando codigo duplicado.

```
Vuelo   (TypeObject)
   ↑                  //yo soy un servicio que pertenece al vuelo AR1552
Servicio (Object)    //no crea subclases para cada tipo, el tipo es un objeto ahora
```

```
//y cuando necesita algo
servicio.getOrigen()  →  servicio.vuelo.getOrigen()
```

!!Entonces, Type-Object aparece cuando en el dominio hay algo que parece una clase pero en realidad es un concepto de datos compartidos

//Ejemplo video 2



En el dominio hay dos niveles distintos:

El Titulo: Representa autores, título, cantidad de páginas, editorial, año. Estas cosas son iguales para TODAS las copias.

La Copia: Tiene un ID unico, historial de prestamos, quien la tiene ahora. Esto cambia entre copias.

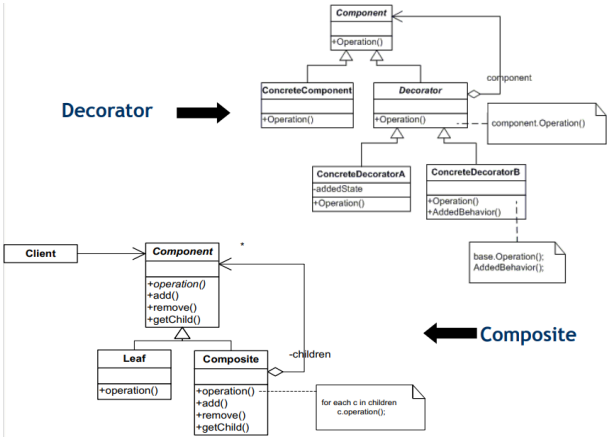
El problema es que la copia debe guardar todo y, por lo tanto, se tendra datos duplicados/ desperdicio de memoria.

Solución: Separar. El titulo describe el tipo y la copia representa la instancia concreta. “Yo soy una instancia del título X”.

= COMPARACIONES =

Relación entre patrones estructurales

Decorator vs Composite →



Aunque ambos patrones comparten una estructura similar y ambos se basan en la composición recursiva para organizar un número indeterminado de objetos, su **propósito y uso es distinto**.

Similitudes y Colaboración

Transparencia: En ambos patrones, las clases (decoradores o compuestos) heredan de la misma clase abstracta que los objetos que contienen. Esto permite que el cliente no necesite saber si está tratando con un objeto simple, un objeto decorado o un compuesto
Complementariedad: Son patrones complementarios. Es muy común usar un **Decorator** para añadir responsabilidades (como un borde) a un objeto que internamente es un **Composite** (una agrupación de elementos gráficos). Dado que el Decorator es transparente, el Composite no necesita saber que ha sido decorado.

Diferencias

- **Propósito**: Composite tiene como objetivo **representar jerarquías de parte-todo** y permitir que los clientes traten de manera uniforme tanto a los objetos individuales como a los compuestos.

  Decorator es diseñado para **añadir responsabilidades a un objeto dinámicamente** sin recurrir a la herencia (subclases). Evita la explosión de subclases que ocurriría si intentáramos cubrir todas las combinaciones de responsabilidades/funcionalidades mediante herencia estática.

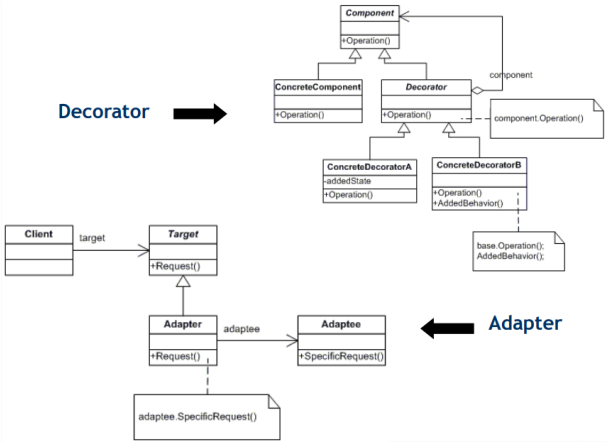
- **Relación de Objetos**: El **Composite** está diseñado para agregar **múltiples componentes**. Su clases definen operaciones para gestionar hijos (Añadir, Eliminar, ObtenerHijo), algo que no es el foco del Decorator.

  El **Decorator** suele tener **un solo componente** hijo. Se puede ver como un "Composite degenerado" que solo contiene un elemento. Sin embargo, su función no es la agregación de objetos, sino añadir funcionalidades (como dibujar un borde o añadir una barra de desplazamiento) antes o después de reenviar la petición al componente.

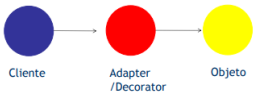
- **Responsabilidad:** El **Composite** simplemente propaga las peticiones a sus hijos para que realicen las operaciones, o las realiza él mismo basándose en sus hijos, pero su fin es tratar a muchos objetos como uno solo.

  El **Decorator** añade responsabilidades adicionales (comportamiento) que no están presentes en el componente original. Está diseñado para permitir añadir funcionalidades sin crear subclases.

Decorator vs Adapter →



Aunque ambos patrones se conocen también bajo el nombre de Wrapper (Envoltorio) ya que envuelven un objeto dentro de otro, su proposito es distinto.



Similitudes

- Ambos patrones envuelven un objeto para cambiarlo.
- Ambos interceptan mensajes enviados por el cliente.
- Ambos permiten modificar el comportamiento del objeto envuelto.

Diferencias

- **Interfaz:** Decorator cambia las responsabilidades del objeto original pero preserva la interfaz. El decorador debe ajustarse exactamente a la interfaz del componente que decora para ser transparente ante el cliente.

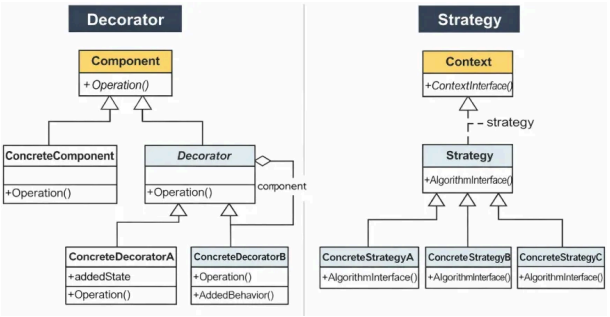
  Adapter tiene como función principal cambiar la interfaz del objeto, convierte la interfaz original (Adaptable) en una interfaz completamente nueva (Objetivo) que es la que esperan los clientes.

- **Propósito**: Adapter se utiliza cuando se tiene una clase existente que funciona correctamente, pero su interfaz no coincide con la que necesitas en tu sistema actual. Su objetivo es hacer que clases incompatibles trabajen juntas.

Decorator se utiliza cuando quieres añadir funcionalidades o características a un objeto de forma dinámica (en tiempo de ejecución) sin afectar a otros objetos de la misma clase y sin recurrir a la creación de subclases estáticas (herencia).

- **Transparencia:** El **Decorator** está diseñado para ser transparente; el cliente no debería necesitar distinguir entre el componente original y el decorado, ya que ambos comparten la misma interfaz. El **Adapter** no busca esta transparencia del mismo modo, ya que explícitamente presenta una cara distinta (la interfaz Objetivo) para ocultar y adaptar la interfaz original (Adaptable).
- **Recursividad:** Debido a que el Decorator mantiene la misma interfaz, se pueden anidar múltiples decoradores recursivamente para añadir múltiples responsabilidades. El Adapter no está pensado para esta composición recursiva, ya que su fin es simplemente conectar dos interfaces distintas.

Decorator vs Strategy →



El próposito de ambos patrones es muy similar, permiten que un objeto cambie su funcionalidad dinámicamente, ya sea agregando comportamiento o modificando el algoritmo que utiliza.

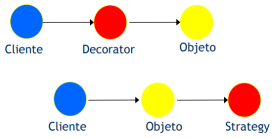
Hay una analogía visual muy clara que utilizan los autores: *el Decorator cambia la "piel" de un objeto, mientras que el Strategy cambia sus "tripas".*

Diferencias

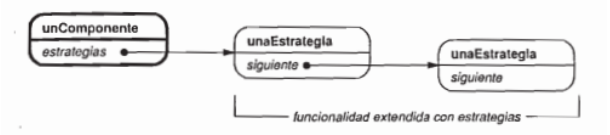
- **Propósito**: Decorator tiene como objetivo **añadir responsabilidades** o funcionalidades a un objeto desde fuera, es decir, cambia el comportamiento desde el exterior del objeto. Se utiliza para extender lo que el objeto puede hacer, sin modificar su código interno ni la clase original.

  Strategy tiene como objetivo cambiar el algoritmo/funcionamiento interno desde el interior del objeto. Se utiliza cuando una clase quiere variar *cómo* realiza una tarea específica delegando esa tarea en un objeto estrategia intercambiable.

- **Estructura y delegación**: El **Decorator** *envuelve* al objeto original. Los decoradores son transparentes para el cliente y se pueden apilar recursivamente (un objeto dentro de un decorador, que está dentro de otro, etc.) para sumar múltiples funcionalidades
  El **Strategy** es un objeto *delegado*. El objeto principal (Contexto) tiene una referencia a una instancia de Estrategia y le delega una parte de su lógica interna.



- **Relación de los componentes con sus extensiones:** En Strategy, el componente conoce sus posibles extensiones ya que necesita delegar cierta lógica y, por lo tanto, debe referenciar y mantener las estrategias correspondientes.



En Decorator el componente no tiene que saber nada de sus decoradores; es decir, los decoradores son transparentes para el componente.

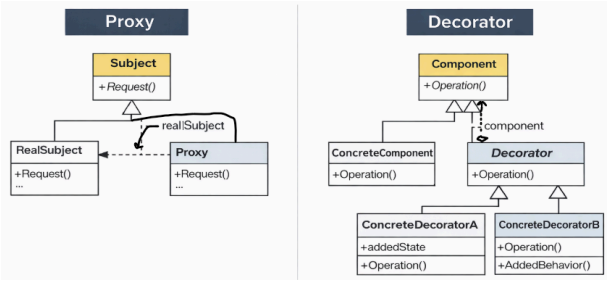
- En Strategy, agregar nuevas estrategias puede requerir modificar el componente. En Decorator no seria necesario.
- En Strategy una estrategia puede tener su propia interfaz especializada, mientras que la interfaz de un decorador debe ajustarse a la interfaz del componente.

¿Cuándo elegir cada uno?

Es mejor usar **Strategy** si la clase `Componente` es pesada o compleja, ya que hace que el patrón Decorator pueda resultar costoso de aplicar (en términos de cantidad de clases y objetos), y se necesita variar su comportamiento interno sin hacerla aún más grande.

Caso contrario, ante una clase `Componente` ligera y se quiere añadir funcionalidades variadas de forma dinámica y transparente, , sin que el componente tenga que saber nada sobre esas extensiones, es mejor usar Decorator.

Decorator vs Proxy →



Aunque el patrón **Decorator** (Decorador) y el patrón **Proxy** (Apoderado) tienen estructuras muy similares (ambos actúan como intermediarios que envuelven a otro objeto), su proposito y el grado de responsabilidad es distinto.

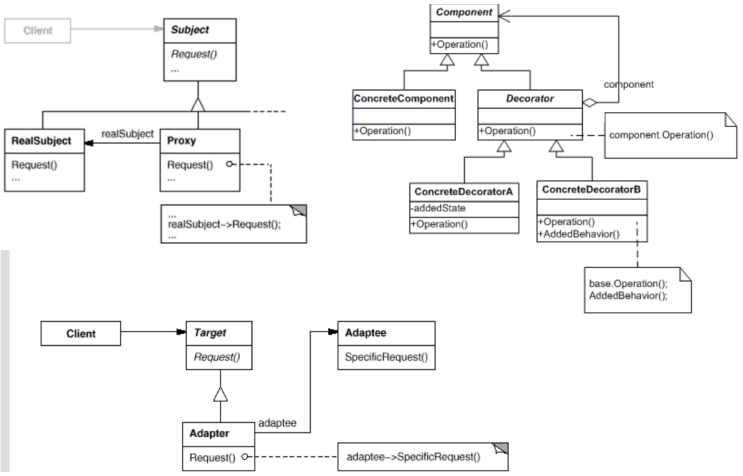
Similitudes →

- Ambos patrones mantienen una referencia a otro objeto y le reenvían peticiones
- Tanto el decorador como el proxy deben implementar la misma interfaz que el objeto al que envuelven. Esto es crucial para que el cliente no sepa si está tratando con el objeto real, con un proxy o con un decorador

Diferencias →

- **Propósito**
- **Funcionalidad:** Decorator añade una o más responsabilidades al objeto que envuelve. En Proxy, el sujeto real define la funcionalidad principal y el proxy simplemente proporciona acceso a ella, no suele añadir funcionalidad "de negocio", si no funcionalidad de gestión (creación, borrado, control de acceso),
- **Gestión del objeto real:** En Decorator el componente ya existe y se le pasa al decorador para que lo envuelva. El decorador no suele ser responsable de crear el componente, solo de adornarlo. En Proxy, el proxy es responsable de **crear y borrar** el objeto real.

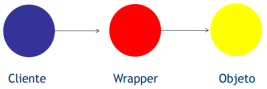
Comparación entre aquellos patrones estructurales llamados wrapper (Adapter, Decorator, Proxy)



Aunque los tres patrones se basan en una estructura similar donde un objeto "envuelve" a otro (guardando una referencia a él y redirigiéndole peticiones), tienen sus diferencias.

Similitudes

- Todos los diagramas de objetos son similares, en los tres patrones: existe un objeto "intermediario" que guarda una **referencia a otro objeto** (composición), y este intermediario intercepta las peticiones del cliente y las **reenvía (delega)** al objeto interno.



obs. Decorator puede tener más de un wrapper, pero suponiendo que hay solo uno, se ven todos así.

Diferencias

- **Propósito**
  - **Adapter:** Es una caja que tiene un enchufe redondo por fuera pero conecta con un enchufe cuadrado dentro. (Cambia la conexión).
  - **Decorator:** Es una caja pintada de colores y con lazos que envuelve el regalo original. (Añade adorno/función sin cambiar lo de adentro).
  - **Proxy:** Es una caja fuerte transparente o una foto de la caja. (Controla quién puede abrirla o representa la caja real hasta que esta llega).
- **Relación con la Interfaz:**
  - (Diferente) El adaptador presenta una interfaz *diferente* a la del objeto envuelto (la del *Adaptee*). Presenta la interfaz del *Target* (Objetivo).
  - (Idéntica) El decorador debe tener la *misma* interfaz que el objeto envuelto para ser transparente ante el cliente.
  - (Idéntica) El proxy presenta la *misma* interfaz que el sujeto real para que el cliente no sepa que está tratando con un sustituto.

Relación entre patrones de comportamiento

Relación entre Strategy y otros patrones

Strategy vs Template Method →

Similtudes →

- Ambos patrones **reducen la rigidez del código** separando las partes variables de un algoritmo de las partes que permanecen constantes.

Template Method permite variar algunos pasos dentro de un algoritmo fijo y Strategy permite cambiar el algoritmo que utiliza un objeto

- **Ambos ayudan a organizar el código y evitar la duplicación:** El template Method es fundamental para la "refactorización" de código, moviendo la parte invariante a una clase padre común.

  Strategy elimina las sentencias condicionales complejas (if/switch) que seleccionan comportamientos, moviendo cada rama a su propia clase.

Diferencias →

- **Mecanismo: Herencia vs Composición**
  - Template Method utiliza la herencia para introducir variabilidad. Se define el "esqueleto" del algoritmo en una clase abstracta y las subclases especializan ciertos pasos redefiniendo métodos. Como la variación se logra creando subclases y la estructura no puede cambiar mientras el programa esta funcionando ya que la herencia se define en tiempo de compilación, se dice que Template Method usa una relación estática, para cambiar el algoritmo se tiene que recompilar el programa.
  - Strategy utiliza la composición de objetos (delegación) para introducir variabilidad. Se define una interfaz común (Strategy) y las estrategias concretas implementan distintos algoritmos. El contexto mantiene una referencia a una estrategia concreta para delegar en ella la ejecución del algoritmo, lo que permite cambiar el algoritmo dinámicamente en tiempo de ejecución sustituyendo un objeto estrategia por otro.
- **Parte vs Todo**
  - Template Method varía una parte del algoritmo. Fija la estructura del algoritmo (el orden de los pasos) en una clase abstracta y permite variar la implementación de algunos pasos mediante subclases.
  - Strategy varía el algoritmo completo. Encapsula una familia de algoritmos en objetos independientes e intercambiables, permitiendo cambiar el algoritmo sustituyendo una estrategia por otra.

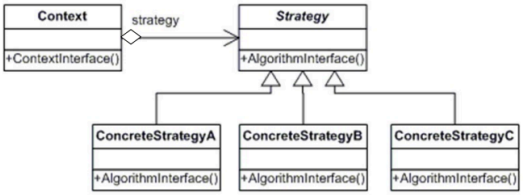
▸ **Ambos patrones pueden combinarse**

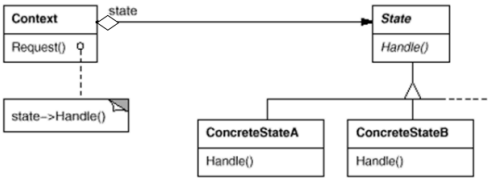
A menudo, las clases Strategy se implementan internamente utilizando el patrón Template Method.

¿En qué situación? Cuando existen variantes de comportamiento (Strategy) pero cada variante sigue una secuencia de pasos fija (Template Method)

Por ejemplo, hay distinta formas de pagar, por tarjeta de credito, de debito, etc. Si hay una serie de pasos en pagar, como verificar los datos y efectuar el pago, sería necesario combinar ambos patrones.

Strategy vs State →





Similitudes →

- Estructura
  - Ambos patrones comparten la misma estructura base, en ambos casos, existe una clase Contexto que mantiene una referencia a una clase abstracta/interfaz y delega las operaciones específicas en ella.

Diferencias →

- Propósito
  - El objetivo de State es permitir que un objeto modifique su comportamiento cuando su estado interno cambia.
  - El objetivo de Strategy es permitir que un cliente elija entre diferentes algoritmos para realizar una tarea específica
- Quien decide el cambio de comportamiento
  - En State, el cliente interactua únicamente con el Contexto y no elige directamente el estado directamente ni necesita conocer cúal está activo internamente.
  - En Strategy el cliente conoce las distintas estrategias disponibles y decide cual utilizar para luego configurar el Contexto pasándole la estrategia concreta.

    El contexto debe ofrecer un método público (setStrategy) que permite cambiar dinámicamente la estrategia si es necesario.
- Visibilidad y transiciones
  - En State el estado es interno y encapsulado dentro del objeto Contexto, el cliente no conoce cual estado concreto está activo. La lógica de transición de estado puede estar implementada dentro de los propios estados concretos (que pueden conocerse entre sí) o dentro del Contexto.
  - En Strategy el cliente conoce qué estrategia está usando el Contexto y puede decidir cambiarla. No existe el concepto de "transición" entre estrategias; las estrategias concretas son independientes entre sí (no se conocen) y el cambio suele ser explícito.
- Cantidad de mensajes
  - En State se puede definir múltiples métodos, cuyo comportamiento varía según el estado actual. El mismo mensaje puede producir resultados diferentes dependiendo del estado.
  - En Strategy hay un algoritmo definido por la interfaz que suele modelarse con un método principal. Sin embargo, podría haber más métodos relacionados.

Comparación entre patrones de comportamiento

Similitud principal→

- **Encapsular lo que varía**

Estos patrones buscan encapsular y aislar aquellos aspectos de un sistema/programa que pueden cambiar con frecuencia, separándolos de aquello que permanece constante.

Definen una clase abstracta o interfaz que encapsula dicho aspecto, es decir, representa el comportamiento variable.

→ Straegy: encapsula un algoritmo dentro de objetos estrategia intercambiables.

→ State: encapsula un comportamiento que depende del estado interno en objetos estado.

→ Template method: encapsula la estructura (esqueleto) del algoritmo en una clase abstracta base y permite varias pasos específicos mediante herencia.

Relación entre patrones creacionales

Diferencia entre Factory Method y Builder

- Propósito
  - Factory Method: Define una interfaz para crear un objeto, pero permite que las subclases decidan qué clase instancia.Su objetivo principal es permitir que una clase delegue la responsabilidad de la creación a sus subclases.
  - Builder: Se utiliza para separar la construcción de un objeto complejo de su representación. Esto permite que el mismo proceso de construcción pueda dar lugar a diferentes representaciones finales del producto
- Proceso de Construcción

- Construcción en un solo paso: Factory Method crea el objeto en un solo paso y lo devuelven inmediatamente al cliente
- Construcción incremental: El patrón Builder construye el producto paso a paso bajo el control de un objeto "Director". El producto final no se devuelve inmediatamente, sino que se recupera del constructor una vez finalizado todo el proceso.

  La estructura es la misma, pero el resultado final varía según el ConstructorConcreto que se utilice, permitiendo que las partes y su representación final sean distintas.

- **Ámbito del Patrón**
  - Patrón de Clases: El Factory Method es un patrón de creación basado en clases, lo que significa que utiliza la herencia para variar el objeto instanciado.
  - Patrón de Objetos: El Builder es un patrón de creación basado en objetos, ya que delega la creación a otro objeto (el constructor concreto) mediante la composición.

	FactoryMethod	Builder
Problema	Construcción	Construcción
Encapsula complejidad al cliente?	si/no	si/no
Definición unica ? (receta)	si/no	si/no
Identifica proveedor de partes?	si/no	si/no
Simil constructor?	si/no	si/no
Primer estadio de re-diseño? (usualmente)	si/no	si/no

› Encapsula complejidad al cliente: Ambos separan al cliente de detalles de creación.

› Definición única (receta): El factory method solo define que crear, no es como el Builder que define los pasos.

› Identifica proveedores de partes: ¿El patrón deja en claro quién es el responsable de crear (o proveer) los objetos?

› Factory Method delega la decisión sobre qué producto concreto crear a ConcreteCreator, pero no construye el objeto por partes ni identifica proveedores de componentes internos (a diferencia de Builder)

› Similar constructor: Factory method si crea el objeto pero Builder no lo hace directo, separa la cr

› Primer estadio de re-diseño: Forma de ordenar cuándo suele aparecer cada patrón al mejorar un diseño. Cuando un sistema empieza a crecer, el primer problema es “esta clase esta llena de new y depende de clases concretas”.

› Factory Method aparece antes, como una solución simple. Se usa cuando se quiere encapsular la creación de objetos.

› Builder aparece despúes, cuando la construcción ya es compleja.

Como diferenciar Strategy con State/Decorator

Si hay una ocasión en la que no hay necesidad de tener un comportamiento/decorador o siempre está la necesidad de elegir entre uno u otro. En Strategy solo hay un comportamiento que se delega a la estrategia, a diferencia del state donde el contexto delega prácticamente todo a su estado porque no sabe como resolver practicamente nada ya que todo depende del estado.

El state se aplica más cuando hay transición entre estados.

Frameworks

Refactoring

Testing

Ejemplo de refactoring